[Lung Disease Classification from Chest X-Rays]

Emre Dumbo, 99990935744

Department of Computer Engineering, emredumbo@ogr.eskisehir.edu.tr

Buğra Muhçi, 14492562250

Department of Computer Engineering, bugramuhci@ogr.eskisehir.edu.tr

***Abstract*—This project focuses on the classification of grayscale images into 6 categories using deep learning techniques. Initially, the NIH Chest X-rays dataset [1] [13] contained 112,000 images across 14 classes. Through a cleaning and preprocessing process, this dataset was reduced to 19,561 images, narrowing down the focus to 6 important categories. A custom deep learning pipeline using the ResNet50 [4] [13] architecture was developed to preprocess, augment, and train the model. The preprocessing steps included normalizing grayscale images and using class weight balancing to address the issue of data imbalance. In addition, techniques like label smoothing and dynamic learning rate scheduling were applied to improve model performance. The final model achieved a validation accuracy of 55%, demonstrating its capability in real-world classification tasks. This project highlights skills in machine learning, model optimization, and the importance of efficient data preparation to achieve meaningful results.**

***Index Terms*—Machine learning, deep learning, image classification, grayscale image processing, ResNet50, class imbalance, data preprocessing, TensorFlow, accuracy.**

I. Introduction

Lung diseases such as pneumothorax and atelectasis are significant health challenges worldwide [8]. Chest X-rays are commonly used for diagnosing these conditions, but interpreting them requires specialized knowledge, which is not always available, especially in areas with limited resources [9]. Machine learning has emerged as a potential solution to automate this process, offering faster and more accurate diagnoses.

In this project, the goal was to classify lung diseases from chest X-rays using 4 different machine learning models: MobileNetV3, EfficientNet, ResNet50, and a custom Convolutional Neural Network (CNN) [4]. Among these, ResNet50 yielded the best validation accuracy, demonstrating its effectiveness for this task. This study provided practical experience with machine learning techniques and insights into how variations in model architecture and training strategies impact performance.

The dataset and the trained ResNet50 Model used in this study is accessible via Google Drive at this link [13].

II. Proposed Methodology

In this project, we aim to classify grayscale images into 6 categories using machine learning techniques, focusing on image classification tasks. The methodology consists of 4 main components: model selection, implementation strategy, evaluation metrics, and testing setup.

A. Model Selection

We used **ResNet50**, a deep convolutional neural network (CNN) architecture known for its residual connections, which help mitigate the vanishing gradient problem and allow for deeper learning. ResNet50 has been successfully applied in various image classification tasks, making it an ideal choice for our problem. Additionally, we leveraged **transfer learning**, utilizing a pre-trained ResNet50 model on ImageNet to benefit from learned features, thus reducing the need for extensive training time and computational resources.

B. Implementation Strategy

The first step involved data preprocessing. Grayscale images were converted to RGB format to match the input requirements of ResNet50. The images were resized to 224x224 pixels, normalized, and augmented using techniques such as random rotations, zooms, and shifts [6]. This augmentation enhanced the generalization ability of the model and prevented overfitting. The ResNet50 model was fine-tuned by removing its top layers and adding custom layers specific to our 6-class classification task. These layers included a **Global Average Pooling** layer, a **dropout** layer for regularization, and a **dense** layer with a **softmax** activation function to predict class probabilities [5] [6] [7].

C. Evaluation Metrics

The model's performance was primarily evaluated using **accuracy**, which indicates the proportion of correct predictions. To assess the model's ability to differentiate between classes, we also used **AUC (Area Under the Curve)**. Additionally, considering potential class imbalances, we evaluated the model using **precision**, **recall**, and **F1-score**. These metrics provided a more comprehensive understanding of the model's performance in various class distribution scenarios, ensuring that each class was handled appropriately.

D. Testing Setup

The project was implemented in **Python** [12] using **TensorFlow** and **Keras** [10] as the machine learning frameworks. The model was trained on a system equipped with an **NVIDIA GeForce RTX 3060 Laptop GPU**, providing 3472 MB of memory, ensuring faster processing for training. The training environment was managed through **Anaconda** in a Python 3.10 environment [11]. The dataset was stored locally, with

separate directories for the training, validation and test sets. The validation set was used to assess the model's generalization ability and prevent overfitting.

This methodology combined pre-trained models with robust data preprocessing, class-weight balancing, and careful evaluation metrics to efficiently and accurately classify grayscale images into 6 categories. Through this approach, we demonstrated the effectiveness of deep learning for real-world image classification tasks.

III. Experiments and Results

In this section, we present the results of our lung disease classification experiments. The performance of the ResNet50 model was evaluated using multiple metrics, as shown in Table I. Table II provides class-wise sensitivity and specificity, which are critical in medical projects. Additionally, Figure 1 illustrates the ROC-AUC curve for the classification task, with the reference to it explained below the image.

A. Performance Metrics

TABLE I
PERFORMANCE METRICS FOR RESNET50 MODEL

Metric	Value
Accuracy	0.55
Precision (Macro)	0.56
Precision (Weighted)	0.58
Recall (Macro)	0.61
Recall (Weighted)	0.55
F1 Score (Macro)	0.56
F1 Score (Weighted)	0.55
ROC-AUC (One-vs-Rest, Macro)	0.87
Balanced Accuracy	0.55

B. Class-Wise Metrics

TABLE II
CLASS-WISE SENSITIVITY AND SPECIFICITY

Class	Sensitivity	Specificity
Atelectasis	0.47	0.93
Cardiomegaly	0.85	0.96
Effusion	0.65	0.92
No Finding	0.53	0.85
Nodule	0.40	0.94
Pneumothorax	0.77	0.86
Average (Macro)	0.61	0.91

C. Visual Representation

The tables and the ROC-AUC curve highlight the performance of our model. Despite a relatively modest overall accuracy (55%), the high AUC scores, particularly for critical classes like Cardiomegaly (0.98) and Pneumothorax (0.90), demonstrate the potential for practical diagnostic applications. Addressing sensitivity and specificity imbalances, especially in underrepresented classes, remains crucial for further improving model performance.

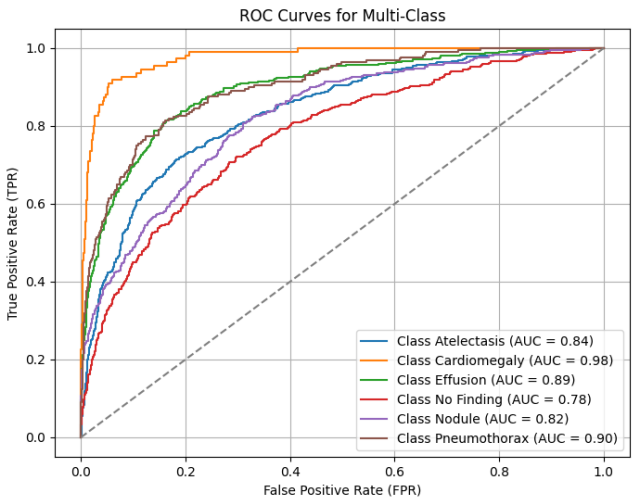

Fig. 1. ROC-AUC Curve for the Classification Task. This curve demonstrates the model's ability to distinguish between classes, with individual AUC values ranging from 0.78 (Class No Finding) to 0.98 (Class Cardiomegaly).

IV. Code Explanation

In this section, we provide a detailed explanation of the code used in the project. The code is divided into several key sections: data preparation, model creation, model compilation, callbacks, and training. Below is a detailed description of each part.

A. Data Preparation and Augmentation

The first step in the code involves importing necessary libraries and setting up data generators for both the training and validation datasets.

- `train_datagen` and `val_datagen` are instances of `ImageDataGenerator` used to perform data augmentation on the training dataset and normalization on the validation dataset [3]. For the training set, random transformations such as rotation, width and height shifts, shear, and zoom are applied to make the model robust and prevent overfitting.
- `train_generator` and `val_generator` are used to load images from the specified directories, resize them to the target image size (`IMAGE_SIZE`), and generate batches of data. The images are scaled by a factor of 1/255 to normalize the pixel values to a range of [0, 1].

B. Class Weights Calculation

The dataset might be imbalanced, meaning some classes may have fewer samples than others. To handle this, class weights are calculated using `compute_class_weight` from `sklearn.utils`. The computed weights are then converted into a dictionary for use during training. This ensures that the model gives more attention to underrepresented classes.

C. Model Creation with ResNet50

The model is built by first loading the pre-trained ResNet50 model without the top layers (i.e., excluding the fully connected layers), which are not needed for our classification task. This pre-trained model is used as a feature extractor.

- `base_model` is initialized with the pre-trained weights from ImageNet. The input shape is set to (224, 224, 3), as required by the ResNet50 model. `include_top=False` ensures that the fully connected layers of ResNet50 are excluded.
- The output of the base model is passed through a `GlobalAveragePooling2D` layer, which reduces the spatial dimensions of the feature maps.
- A `Dropout` layer with a rate of 0.5 is added to regularize the model and prevent overfitting.
- A `Dense` layer with softmax activation is added at the output, which is responsible for classifying the input into one of the six classes [5].

D. Model Compilation

The model is compiled using the Adam optimizer, which is a popular choice for training deep neural networks. The loss function used is `CategoricalCrossentropy` with label smoothing set to 0.1 to help improve model generalization. The performance of the model is evaluated using the accuracy metric and the AUC (Area Under the Curve) metric.

E. Callbacks

Several callbacks are used to improve training and model performance:

- `ModelCheckpoint` saves the best model based on validation accuracy during training.
- `EarlyStopping` monitors the validation accuracy and stops training if the accuracy does not improve for 10 consecutive epochs, helping to avoid overfitting.
- `ReduceLROnPlateau` reduces the learning rate by a factor of 0.5 if the validation loss plateaus, helping the model to converge more effectively.

F. Training the Model

The model is trained using the `fit` function, which takes the training and validation data generators, the number of epochs, class weights, and the defined callbacks as inputs. The model will be trained for a maximum of 30 epochs.

G. Model Evaluation

After training, the model is evaluated on the validation data using the `evaluate` method. This provides the final accuracy and loss metrics, which help assess how well the model generalizes to unseen data [13].

V. Personal Contributions

- **Emre Dumbo**: Emre used his connections to a doctor and a machine learning specialist professor for valuable insights and collaboration. He contributed to code writing and debugging, code optimization, methodology research, and evaluation. Emre was also responsible for dataset selection and contributed a personal chest X-ray as a ”no finding” test image to the dataset.
- **Buğra Muhçi**: Buğra connected with a friend who is a doctor for assistance with class selection from the dataset. He was involved in code writing and debugging, code optimization, prompt engineering, data preprocessing, and model evaluation and selection.

VI. Discussion and Conclusion

In this project, we classified lung diseases from chest X-ray images using deep learning models, specifically ResNet50. Several key observations were made:

A. Model Performance

The ResNet50 model achieved a validation accuracy of 55%. While this result highlights the complexity of medical image classification tasks, the high AUC scores for critical classes such as Cardiomegaly (0.98) and Pneumothorax (0.90) demonstrate the potential for practical applications. Hyperparameter tuning revealed notable variability, with some configurations yielding perfect training accuracy (e.g., 100%) but significantly lower validation accuracy (e.g., 50%), pointing to challenges in generalization. While significant efforts were made to address sensitivity and specificity imbalances for underrepresented classes, these improvements were limited, indicating a need for further exploration in future work.

B. Data Preprocessing and Augmentation

Preprocessing steps, including normalization, rescaling, and data augmentation (e.g., rotations, shift), enhanced the model's generalization ability and mitigated overfitting. These techniques accounted for variations in chest X-ray images and were crucial to achieving stable performance.

C. Handling Class Imbalance

Class weighting effectively addressed class imbalance, ensuring the model did not favor more frequent classes. This strategy improved performance for underrepresented categories, showcasing its importance in medical datasets.

D. Model Selection and Tuning

The ResNet50 architecture was selected for its robust feature extraction capabilities. Fine-tuning, including label smoothing and dynamic learning rate adjustments, improved generalization. However, experiments with various hyperparameter settings demonstrated inconsistent outcomes, occasionally achieving overly high training accuracy with poor validation performance, reflecting a need for more stable optimization strategies. Despite our efforts to explore alternative architectures and hybrid models, performance improvements were incremental, suggesting further research is needed in this area.

E. Future Directions

To improve results, future work could focus on:

- Expanding the dataset size to include more diverse and balanced samples.
- Refining hyperparameter tuning strategies to improve generalization stability and reduce overfitting.
- Exploring advanced techniques such as domain-specific data augmentation, synthetic data generation, or self-supervised pretraining to enhance performance with limited data.

F. Conclusion

This project demonstrates the potential of deep learning in medical image analysis, showing promising results in identifying critical conditions. However, challenges such as dataset size, class imbalance, and generalization stability remain significant hurdles. The results underscore the need for continuous advancements in data quality, model design, and fine-tuning techniques to improve performance. Future work should focus on expanding datasets, refining model architectures, and exploring domain-specific pre-training to enhance the applicability of deep learning models in real-world healthcare settings.

VII. Ethical Statement

We confirm that all experimental work in this project was carried out by us with integrity. All sources and methods used in the study have been properly cited and included in the references. No plagiarism has been committed in any part of this study.

VIII. AI Tools Usage

In this project, the main AI tool used was ChatGPT [2], which provided assistance in various aspects of the work. Specifically, the AI tool was used for:

A. Code Writing and Debugging

ChatGPT helped generate code snippets, suggest optimizations, and offered solutions to issues encountered during development. This contributed to speeding up thze coding process and improving code quality.

B. Code Optimization

The model provided suggestions for optimizing the code, including adjusting certain parameters and refining the implementation to improve performance and efficiency.

C. Methodology Clarifications

Whenever clarification was needed on methodology or machine learning concepts, ChatGPT provided concise explanations, helping to ensure that the project's approach was well-understood and properly framed.

Although ChatGPT assisted in various technical and writing-related tasks, it is important to note that the conceptual work, data collection, model selection, and decision-making were entirely handled by the team. The contributions of ChatGPT were meant to support the team's efforts and enhance the overall quality of the project, but the primary responsibility for the work lies with the team members.

References

[1] Kaggle, "NIH Chest X-rays Dataset," https://www.kaggle.com/datasets/nih-chest-xrays/data, [Accessed: Dec. 2024].

[2] OpenAI, "ChatGPT," https://chat.openai.com, [Accessed: Dec. 2024].

[3] P. Loukari, "NIH Chest X-rays Classification," GitHub repository, https://github.com/paloukari/NIH-Chest-X-rays-Classification/tree/master, [Accessed: Dec. 2024].

[4] Keras, "Keras API Documentation," https://keras.io/api/, [Accessed: Dec. 2024].

[5] M. Koç, Associate Prof. Dr., Computer Science Branch, Department Chair, Eskişehir Technical University, Email: mehmetkoc@eskisehir.edu.tr.

[6] S. Candemir, Asst. Prof. Dr., Computer Science Branch, Department Vice Chair, Eskişehir Technical University, Email: semacandemir@eskisehir.edu.tr.

[7] B. Karahoda, Associate Prof. Dr., Computer Science Branch, UBT Higher Education Institution.

[8] B. Köroğlu, Medical Student, Hacettepe University.

[9] A. Kazazi, Assoc. Prof. Dr., Gazi University Faculty of Medicine.

[10] TensorFlow, "TensorFlow API Documentation," https://www.tensorflow.org/api_docs/python/tf, [Accessed: Dec. 2024].

[11] Anaconda, Inc., "Anaconda Cloud," https://anaconda.cloud, [Accessed: Dec. 2024].

[12] Python Software Foundation, "Python Documentation," https://www.python.org/doc/, [Accessed: Dec. 2024].

[13] Google Drive, "ResNet50 Model and NIH Dataset," https://drive.google.com/drive/folders/1DMyWrGlT2p1CiQlM6K2aXYlUdI-efoyM?usp=sharing, [Accessed: Dec. 2024].